TP 1 — Pratique avec les Requêtes et Réponses HTTP en Utilisant Postman

Ralph Bou Nader

Master — Ingénierie Informatique

Objectifs

Ce TP vous permet de manipuler concrètement les notions vues en cours (Chapitre 2 — Protocoles et HTTP) à l'aide de **Postman**, l'outil de référence pour construire, envoyer et automatiser des requêtes API. À l'issue de ce TP, vous saurez :

- Construire et envoyer des requêtes HTTP (GET, POST, PUT, PATCH, DELETE) avec Postman.
- Manipuler URL, paramètres de requête, en-têtes et corps de requête.
- Interpréter les codes de statut et diagnostiquer des erreurs.
- Utiliser des **variables d'environnement** pour rendre une collection réutilisable.
- Écrire des **tests automatisés** (assertions) directement dans Postman.
- Simuler une authentification par jeton (Bearer token).
- Exporter et documenter une collection Postman.

Prérequis

- Postman installé (application de bureau ou extension navigateur) : https://www.postman.com/downloads/
- Un compte Postman gratuit (pour synchroniser vos collections).
- Une connexion Internet (les API utilisées sont publiques et gratuites).

Mise en place

1. Ouvrez Postman et connectez-vous (ou créez un compte gratuit).
2. Créez une nouvelle **Collection** nommée `API Protocols Practice`.
3. Créez un **Environnement** nommé `JSONPlaceholder Env` contenant une variable :
   - `base_url` = `https://jsonplaceholder.typicode.com`

   Utiliser une variable d'environnement plutôt que de recopier l'URL complète à chaque requête est une pratique professionnelle standard : elle permet de changer d'environnement (développement, test, production) sans modifier une seule requête. Vous appellerez vos URLs sous la forme `{{base_url}}/posts` tout au long de ce TP.
4. Sélectionnez cet environnement dans le menu déroulant en haut à droite de Postman avant de continuer.

Tâche 1 — Comprendre les URLs et la méthode GET

API utilisée : JSONPlaceholder, une API REST gratuite pour les tests et prototypes.

Requête GET simple

1. Créez une requête `GET Posts`, méthode `GET`, URL `{{base_url}}/posts`.
2. Cliquez sur **Send**.
3. Observez : le code de statut (`Status`), le temps de réponse, la taille de la réponse, et le corps JSON retourné.

Questions à consigner dans votre rapport :

- Quel est le code de statut renvoyé, et que signifie-t-il ?
- Combien d'éléments contient le tableau JSON renvoyé ?
- Comment les données sont-elles structurées (clés, types de valeurs) ?

Paramètres de requête — filtrage et pagination

Objectif : comprendre comment la chaîne de requête (vue en cours, Chapitre 2) permet de filtrer et paginer une collection sans changer de méthode ni d'URL de base.

1. Dans l'onglet **Params**, ajoutez le paramètre `userId` avec la valeur `1`. Observez l'URL se construire automatiquement : `{{base_url}}/posts?userId=1`. Envoyez la requête.
2. Remplacez ce paramètre par deux paramètres de pagination : `_page=1` et `_limit=5`. Envoyez à nouveau.

Question à consigner : combien de résultats obtenez-vous dans chaque cas ? En quoi cette approche (chaîne de requête) est-elle préférable à la création d'un endpoint dédié comme `/posts/userId/1` ?

Requête GET sur une ressource unique

1. Créez une requête `GET Post by ID`, URL `{{base_url}}/posts/1`.
2. Envoyez, puis comparez la forme de la réponse (un objet unique) avec celle de la Section 4.1 (un tableau).

Tâche 2 — Travailler avec les en-têtes HTTP

Objectif : observer comment les en-têtes modifient le comportement d'une requête.

1. Sur la requête `GET Posts`, ouvrez l'onglet **Headers** et ajoutez :
   - `User-Agent` : une valeur de votre choix, par exemple `PostmanRuntime-TP1`.
   - `Accept` : `application/json`.
2. Renvoyez la requête et observez l'onglet **Headers** de la **réponse** cette fois (pas de la requête) : notez `Content-Type`, `Content-Length` et tout autre en-tête présent.

Question à consigner : quelle est la différence de rôle entre un en-tête de requête et un en-tête de réponse ? Rappelez, en une phrase, la différence entre `Content-Type` et `Accept` (vue en cours).

Tâche 3 — Requêtes POST, PUT, PATCH et DELETE

Créer une ressource (POST)

1. Créez une requête `POST Create Post`, méthode `POST`, URL `{{base_url}}/posts`.
2. Onglet **Body** → `raw` → `JSON` :

```
{
  "title": "Nouveau post",
  "body": "Ceci est le corps du nouveau post.",
  "userId": 1
}
```

3. Envoyez. Vérifiez le code de statut (`201 Created` attendu) et notez l'`id` attribué par le serveur.

Remplacer une ressource entière (PUT)

1. Créez `PUT Update Post`, URL `{{base_url}}/posts/1`.
2. Corps JSON complet :

```
{
  "id": 1,
  "title": "Post mis à jour",
  "body": "Ce post a été mis à jour.",
  "userId": 1
}
```

3. Envoyez. Code de statut attendu : `200 OK`.

Modifier partiellement une ressource (PATCH)

Absent du TP original — ajouté pour couvrir les cinq méthodes vues en cours.

1. Créez `PATCH Update Title Only`, méthode `PATCH`, URL `{{base_url}}/posts/1`.
2. Corps JSON, un seul champ :

```
{
  "title": "Titre modifié seulement"
}
```

3. Envoyez, et comparez la réponse à celle obtenue avec PUT.

Question à consigner : quelle différence de principe existe entre PUT et PATCH (rappel du cours : idempotence, remplacement complet vs partiel) ? Le corps que vous envoyez ici illustre-t-il bien cette différence ?

Supprimer une ressource (DELETE)

1. Créez `DELETE Post`, URL `{{base_url}}/posts/1`.
2. Envoyez. Code de statut attendu : `200 OK` ou `204 No Content`.

Question à consigner : si `204 No Content` est renvoyé, que contient le corps de la réponse ? Pourquoi ce choix est-il cohérent avec ce qui a été vu en cours ?

Tâche 4 — Codes de statut et gestion des erreurs

Ressource introuvable

1. `GET {{base_url}}/posts/9999`. Observez le code `404 Not Found`.

Méthode non supportée

1. Changez la méthode de la requête précédente en une méthode inventée en tapant `FOO` directement dans le sélecteur de méthode (Postman l'autorise). Envoyez vers `{{base_url}}/posts/1`.
2. Observez le code `405 Method Not Allowed` (ou une erreur équivalente selon le serveur).

Corps malformé

1. Sur une requête `POST`, entrez un corps JSON volontairement invalide (par exemple une virgule finale : `{"title": "Test",}`).
2. Envoyez et observez le code `400 Bad Request`.

Question à consigner : pour chacun des trois cas, expliquez à qui « appartient » l'erreur (client ou serveur) en vous appuyant sur la classification des codes de statut vue en cours (4xx vs 5xx).

Tâche 5 — Automatiser vos vérifications avec des tests Postman

Section entièrement nouvelle par rapport à la version précédente de ce TP. Objectif : dépasser la simple observation manuelle et apprendre à écrire des assertions automatisées, une compétence professionnelle indispensable en intégration continue.

1. Ouvrez la requête `GET Post by ID` (Section 4.3). Allez dans l'onglet **Tests** (ou **Post-response** selon la version de Postman).
2. Ajoutez le script suivant :

```
pm.test("Le code de statut est 200", function () {
  pm.response.to.have.status(200);
});

pm.test("La réponse contient un champ id", function () {
  const data = pm.response.json();
  pm.expect(data).to.have.property("id");
});

pm.test("Le temps de réponse est inférieur à 1000 ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

3. Cliquez sur **Send** et observez l'onglet **Test Results** : chaque assertion doit apparaître en vert.
4. Ajoutez, sur la requête `POST Create Post`, un test qui vérifie que le code de statut est bien `201` et que la réponse contient un `id`.

Question à consigner : en quoi ces tests automatisés sont-ils plus fiables qu'une vérification visuelle du corps de la réponse ? Dans quel contexte professionnel (rappel : Chapitre 4, gestion des erreurs) ce mécanisme devient-il indispensable ?

Tâche 6 — Simuler une authentification par jeton

Section nouvelle, en préparation du chapitre Sécurité. JSONPlaceholder n'exige pas d'authentification ; nous allons donc simplement construire la requête *comme si* elle en avait besoin, pour manipuler la syntaxe.

1. Sur n'importe quelle requête, ouvrez l'onglet **Authorization**.
2. Sélectionnez le type `Bearer Token` et entrez une valeur arbitraire, par exemple `demo-token-12345`.
3. Envoyez la requête, puis ouvrez l'onglet **Headers** de la requête envoyée (bouton *Code* ou aperçu de la requête brute) pour observer l'en-tête généré automatiquement par Postman : `Authorization: Bearer demo-token-12345`.

Question à consigner : retrouvez, dans l'en-tête généré, la syntaxe exacte vue en cours pour transporter un jeton. Pourquoi ce mécanisme nécessite-t-il HTTPS pour être réellement sûr (rappel du cours) ?

Tâche 7 — Exporter et documenter votre collection

1. Dans le menu de la collection `API Protocols Practice`, sélectionnez **Export** et choisissez le format `Collection v2.1`.
2. Explorez le bouton **Documentation** généré automatiquement par Postman à partir de vos requêtes.

Question à consigner : en quoi cette documentation générée automatiquement se rapproche-t-elle de la notion de contrat d'API vue au Chapitre 1 ?

Rédaction du rapport

Votre rapport doit inclure :

1. Pour chaque tâche : une capture d'écran de la requête et de la réponse dans Postman.
2. Les réponses rédigées à toutes les questions « à consigner » posées dans ce document.
3. Un tableau récapitulatif des méthodes HTTP utilisées, avec pour chacune : l'URL appelée, le code de statut obtenu, et une phrase expliquant le résultat.
4. Une courte réflexion (10 à 15 lignes) sur le rôle de HTTP dans la conception d'API robustes, et sur l'apport des tests automatisés (Tâche 5) par rapport à une vérification manuelle.
5. Le fichier `.json` de votre collection exportée (Tâche 7).

Soumission

Déposez, sur la plateforme du cours, une archive contenant : votre rapport (PDF), la collection Postman exportée (`.json`), et l'environnement exporté (`.json`).

Barème indicatif

Critère	Points
Tâches 1 à 4 (bases HTTP) réalisées et correctement documentées	8
Tâche 5 (tests automatisés) fonctionnelle	4
Tâche 6 (authentification) correctement expliquée	2
Tâche 7 (export et documentation)	2
Qualité et clarté du rapport, réponses aux questions	4
Total	**20**